EECS 348 Assignment 4: UML Use Case Model

Recursive Object-Oriented Sudoku Solver

Nicolas Villalobos

October 15, 2026

1 Use Case Diagram

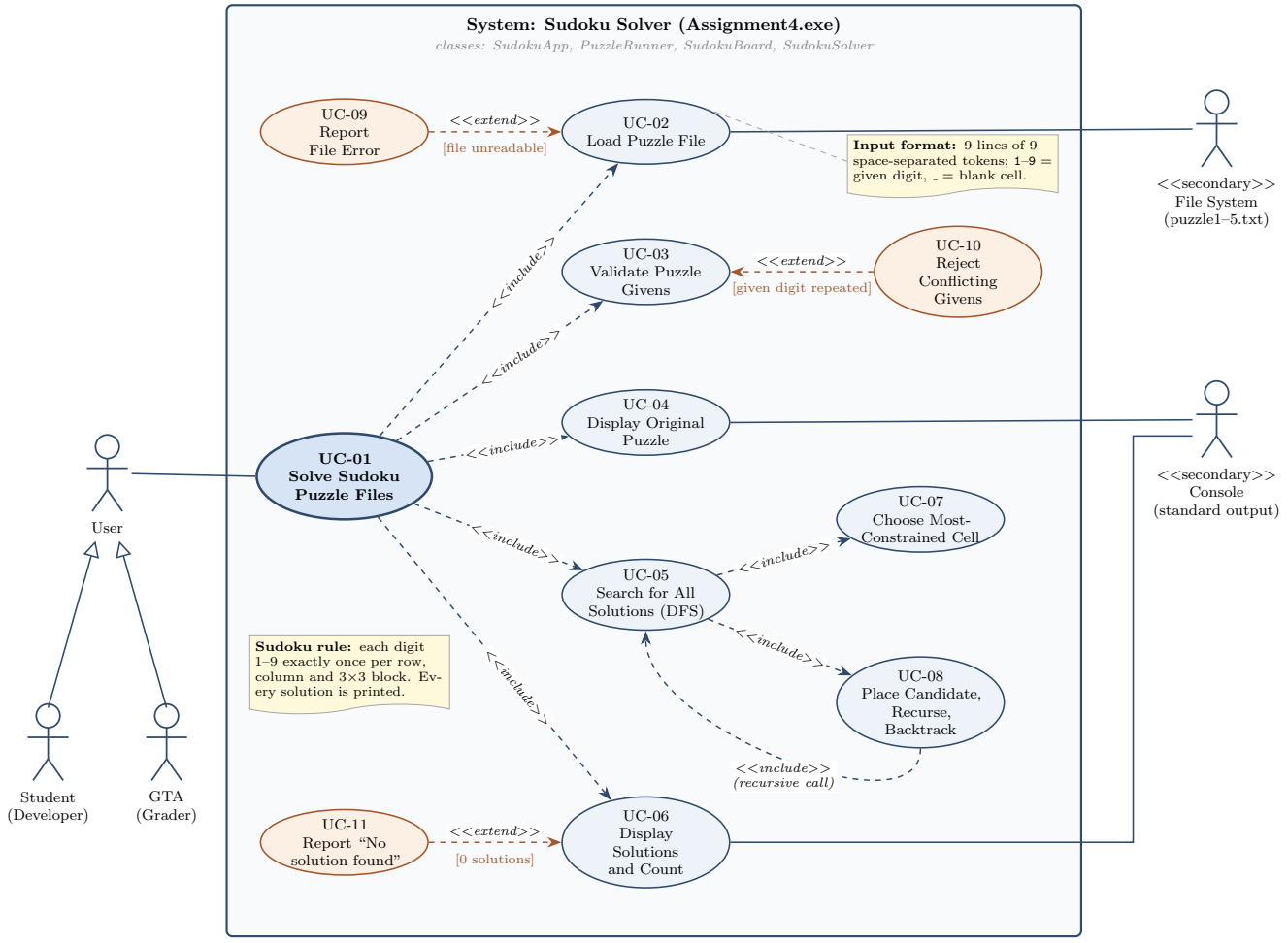


Figure 1: Use case diagram. Blue ovals are the normal path; orange ovals are `<<extend>>` cases that only run when their guard condition holds. Student and GTA both specialize the generic User actor because either one runs the program (the Student while developing, the GTA on the EECS cycle servers when grading).

2 Use Case Identification

Primary use case: UC-01 Solve Sudoku Puzzle Files. The User's goal is to get every valid solution to each puzzle file, or confirmation that none exists. UC-02 to UC-08 are steps UC-01 always performs (`<<include>>`); UC-09 to UC-11 only run when their condition holds (`<<extend>>`).

Actors: User (primary; specialized by Student and GTA), File System (secondary, supplies puzzle1–5.txt), Console (secondary, receives output).

3 Brief Description

The User runs the solver in a directory containing the five puzzle files. For each file the system loads and validates the puzzle, prints the file name and the puzzle as read, then runs a recursive depth-first search with backtracking that tries every legal digit in each empty cell. Each completely filled grid is printed as a solution. At the end the system prints the solution count, or "No solution found" if there were none. Missing or malformed files are reported and skipped.

4 Step-by-Step Flow of Events

1. The User runs `Assignment4.exe`.
2. The system takes the next file (puzzle1.txt to puzzle5.txt) and prints its name.
3. **UC-02:** reads 81 tokens into a 9×9 grid (_ = blank).
4. **UC-04:** prints the puzzle as read.
5. **UC-03:** checks that no given digit repeats in a row, column or block.
6. **UC-05:** recursive search:
   - 6.1 **UC-07:** choose the empty cell with the fewest legal digits; if one has none, backtrack.
   - 6.2 If no empty cell remains, **UC-06** prints the grid as a solution and the search continues.
   - 6.3 **UC-08:** for each legal digit, place it, search again (recursion), then remove it.
7. The system prints "Solutions found: N".
8. Steps 2–7 repeat for each remaining file, then the program exits.

5 Detailed Description

Field	Content
Use case	UC-01 Solve Sudoku Puzzle Files
Actors	Primary: User (Student, GTA). Secondary: File System, Console.
Trigger	User executes `./Assignment4.exe`.
Preconditions	Program compiled on the EECS cycle servers; puzzle1–5.txt in the working directory, each 9 lines of 9 space-separated tokens (`1`–`9` or _).
Postconditions	For each file: name, original puzzle, every solution (numbered) and the count are printed, or "No solution found". Input files are unchanged.
Main flow	Steps 1–8 in Section 4.
Alternate flows	**A1 Multiple solutions:** after printing a solution the search keeps going, so every solution is printed (puzzle5.txt: 2). **A2 No solution (UC-11):** every branch dead-ends; prints "No solution found" (puzzle4.txt). **A3 Conflicting givens (UC-10):** search is skipped, then A2.
Exception flows	**E1–E3 (UC-09):** file missing, fewer than 81 tokens, or an invalid token. The system prints the file name and reason, skips the file, and continues with the next one.
Business rules	Each digit 1–9 appears exactly once per row, column and 3×3 block. All solutions must be printed.
Special requirements	C++11, single file, object-oriented and recursive; runs on the cycle servers. Worst case $O(9^k)$ for k blanks; each provided puzzle solves in under 1 ms.